**SIMATS ENGINEERING**

**CO1 - ASSESSMENT TOOL 1**

**Analytical Problem Solving**

| | |
|---|---|
| **Name of the Student** | **Guggilla Venkatasai** |
| **Reg. No** | **192425388** |

**COURSE NAME: Deep Learning** **COURSE CODE: MLA0403**
**FACULTY : Dr.Arul Raj A.M**

---

**COURSE OUTCOME COVERED**

**CO 1:** Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

**CO1 Weightage: 15%**

**Assessment Tool Weightage: 35%**

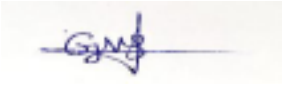
**Duration: 30 minutes**

**Marks: 25**

---

**Code of Conduct:**

I Guggilla Venkatasai(192425388) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



**Signature of the student:**
**Name of the student: Guggilla Venkatasai**

**QUESTIONS:05** **Total Marks:25**

# Analytical Problem Solving

## 1. Learning Algorithms (Optimization Behavior)

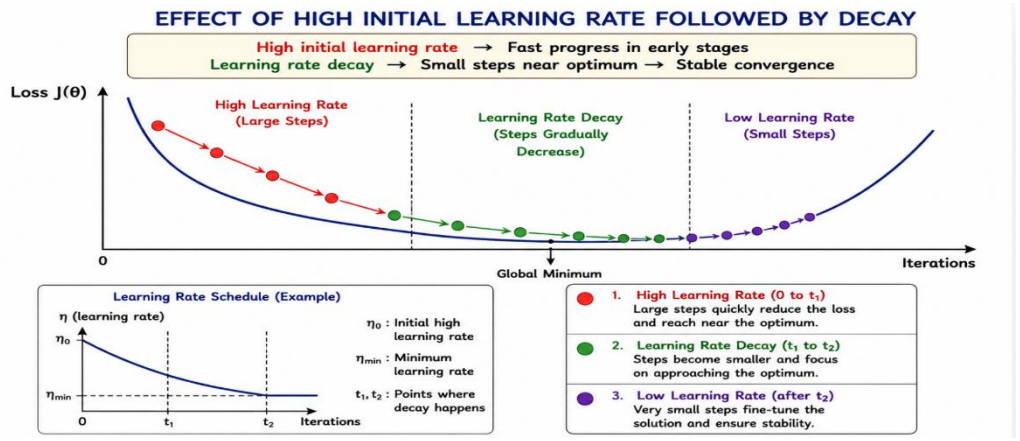
A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

**Question:**
Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer :

Gradient Descent is an optimization algorithm used to minimize the loss function by updating the model parameters in the direction of the negative gradient. The learning rate (η) determines the size of each update. Choosing an appropriate learning rate is important for achieving fast and stable convergence.

When a high learning rate is used initially, the model takes large steps toward the minimum of the loss function. This allows the algorithm to reach the region near the optimum much faster and reduces the number of iterations required during the early stages of training. As training progresses, the learning rate is gradually decreased (learning rate decay) so that the updates become smaller and more precise, allowing the algorithm to converge smoothly to the minimum without oscillating.




**Advantages of High Learning Rate with Decay**

- **Faster Initial Convergence:** Large updates quickly move the parameters toward the optimal region.
- **Improved Training Efficiency:** Reduces overall training time compared to using a small constant learning rate.
- **Stable Final Convergence:** Smaller learning rates near the optimum reduce oscillations and improve accuracy.
- **Better Optimization:** Helps the model reach a lower loss value while maintaining stability.

**When is it Better than a Constant Learning Rate?**

A learning rate with decay performs better than a constant learning rate when:

- The loss function is convex and smooth, allowing large initial steps without instability.
- The initial learning rate is reasonably high but not excessively large.

- The decay schedule gradually reduces the learning rate as the model approaches the optimum.
- Fast convergence is required while still achieving accurate final parameter values.

**Possible Risks**

- If the initial learning rate is too high, the algorithm may overshoot the minimum and oscillate around the optimum.
- Extremely large updates may even cause the training process to diverge, preventing convergence.
- If the learning rate decreases too quickly, the model may stop learning before reaching the optimal solution.
- If the decay is too slow, oscillations may continue near the minimum, reducing convergence efficiency.

**Conclusion**

Using a high initial learning rate followed by gradual decay combines the advantages of fast learning and stable convergence. It often outperforms a constant learning rate by reducing training time and improving optimization. However, the initial learning rate and decay schedule must be carefully chosen to avoid overshooting, divergence, or premature convergence.

## 2. Maximum Likelihood Estimation (MLE)

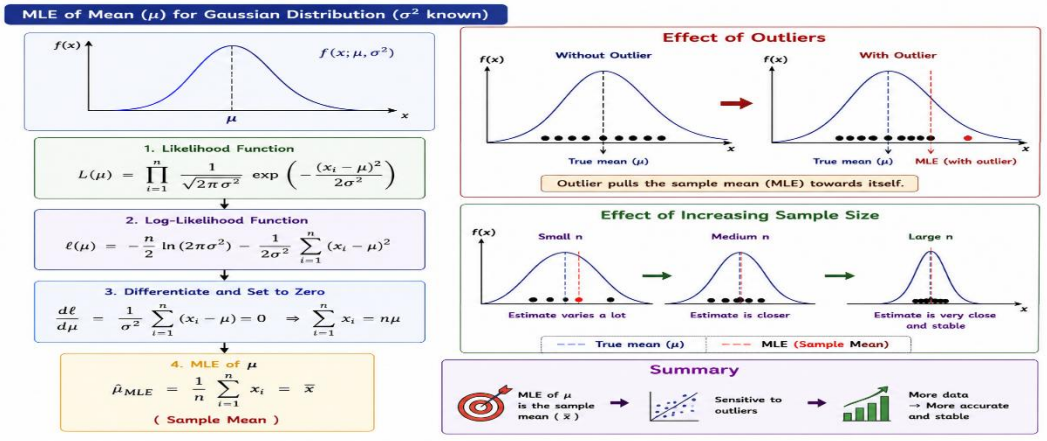
Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance $\sigma^2$

**Question:**
Derive the Maximum Likelihood Estimator for μ, and analytically explain how the estimate changes when:

- The dataset contains outliers
- The sample size increases
  What does this imply about the robustness of MLE?

**Answer:**

Maximum Likelihood Estimation (MLE) is a statistical method used to estimate the parameter values that maximize the probability (likelihood) of observing the given data. For a Gaussian distribution with unknown mean (μ) and known variance ($\sigma^2$), the MLE estimates the value of μ that best explains the observed samples.



## Derivation of MLE for μ

- Let the dataset be:

$$X = \{x_1, x_2, x_3, \dots, x_n\}$$

- The likelihood function for the Gaussian distribution is:

$$L(\mu) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i-\mu)^2}{2\sigma^2}}$$

- Taking the logarithm and differentiating with respect to **μ**, then setting the derivative to zero, we obtain:

$$\boxed{\hat{\mu}_{MLE} = \frac{1}{n}\sum_{i=1}^{n} x_i}$$

- Thus, the **Maximum Likelihood Estimator of μ is the sample mean**.

### Effect of Outliers

An outlier is an observation that is much larger or smaller than the other data points. Since the MLE for μ is the sample mean, even a few extreme values can significantly shift the estimated mean. As a result, the estimate moves toward the outlier and may no longer represent the true center of the data. Therefore, MLE for the mean is sensitive to outliers.

### Effect of Increasing Sample Size

As the number of samples increases, the sample mean becomes more stable and closer to the true population mean. The influence of random variations decreases, resulting in a more accurate and reliable estimate. Hence, the MLE becomes more precise with larger datasets.

### Robustness of MLE

The MLE of the mean performs well when the data follows a Gaussian distribution and contains no significant outliers. However, it is not robust to outliers because extreme observations have a strong influence on the sample mean. Although increasing the sample size improves the accuracy of the estimate, it does not completely eliminate the effect of severe outliers.

### Conclusion

For a Gaussian distribution with known variance, the Maximum Likelihood Estimator of the mean is the sample mean. It provides accurate estimates for clean datasets and becomes more reliable as the sample size increases. However, its sensitivity to outliers makes it less robust in datasets containing extreme values.

## 3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

**Question:**
Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

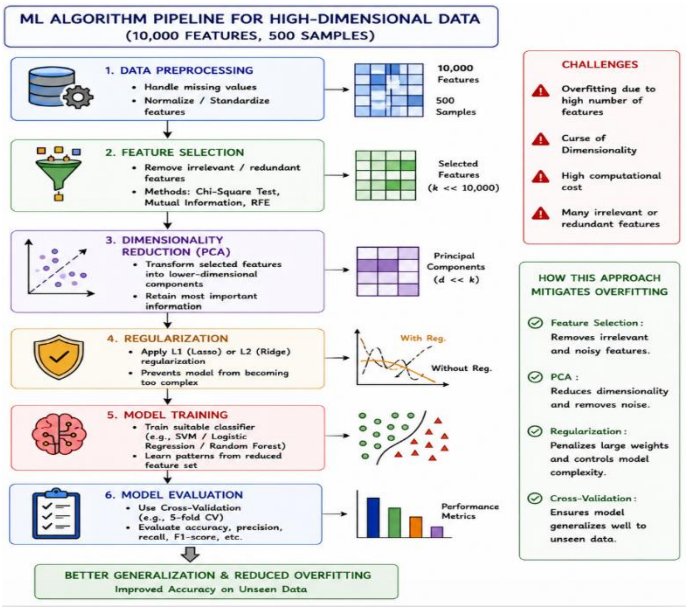
**Answer:**

Training a classification model with 10,000 features and only 500 training samples is a challenging problem because the number of features is much larger than the number of samples. This leads to the curse of dimensionality, where the model becomes complex and learns noise instead of meaningful patterns, resulting in overfitting and poor generalization.

**Challenges**

- **Overfitting:** The model memorizes the training data instead of learning general patterns.
- **Curse of Dimensionality:** High-dimensional data makes it difficult to identify meaningful relationships.
- **High Computational Cost:** More features increase memory usage and training time.
- **Irrelevant Features:** Many features may be redundant or unrelated to the classification task.

**Proposed Algorithm Pipeline**




**Step 1: Data Preprocessing**

- Handle missing values.
- Normalize or standardize the features to bring them to the same scale.

**Step 2: Feature Selection**

- Remove irrelevant and redundant features using methods such as Chi-Square Test, Mutual Information, or Recursive Feature Elimination (RFE).
- This reduces dimensionality while retaining important features.

**Step 3: Dimensionality Reduction**

- Apply Principal Component Analysis (PCA) to transform the selected features into a smaller set of informative components.
- PCA reduces noise and computational complexity.

**Step 4: Regularization**

- Use L1 (Lasso) or L2 (Ridge) regularization to prevent the model from becoming too complex and to reduce overfitting.

**Step 5: Model Training**

- Train a suitable classifier such as Support Vector Machine (SVM), Logistic Regression, or a Random Forest using the reduced feature set.

**Step 6: Model Evaluation**

- Evaluate the model using Cross-Validation to ensure good generalization and reliable performance on unseen data.

**How This Approach Reduces Overfitting**

- Feature Selection removes unnecessary features.
- PCA reduces dimensionality and eliminates noise.
- Regularization limits model complexity.
- Cross-Validation ensures the model generalizes well instead of memorizing training data.

**Conclusion**

For datasets with 10,000 features and only 500 training samples, combining feature selection, dimensionality reduction, regularization, and cross-validation produces a simpler and more efficient model. This approach reduces overfitting, lowers computational cost, and improves classification accuracy on unseen data.

## 4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

**Question:**
Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer:

A Multilayer Perceptron (MLP) is a feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer. The hidden layer uses sigmoid activation functions to learn complex relationships between inputs and outputs.

**Why Adding More Hidden Neurons Increases Representational Power**

Adding more hidden neurons increases the network's ability to learn complex and non-linear patterns. Each hidden neuron learns a different feature of the input data. As the number of neurons increases, the network can represent more complicated decision boundaries and capture intricate relationships among features. This improves the model's capacity to solve difficult classification and prediction problems.

For example, with only a few hidden neurons, the network may learn only simple patterns. Increasing the number of neurons allows the network to identify more detailed and meaningful features, resulting in better representation of the data.

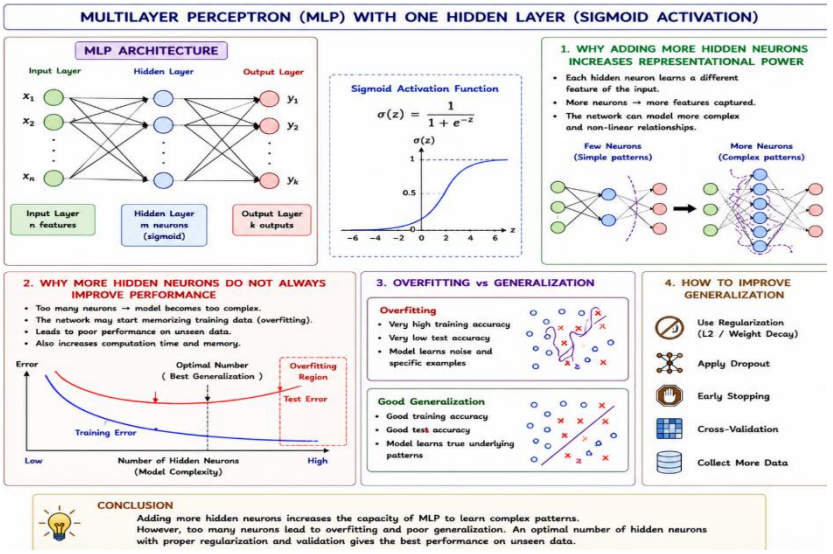
**Why More Hidden Neurons Do Not Always Improve Performance**

Although adding more neurons increases learning capacity, it does not always improve performance. If the network has too many hidden neurons compared to the amount of training data, it may overfit the training data. Overfitting occurs when the model memorizes the training examples instead of learning general patterns, leading to poor performance on unseen data.

A larger network also requires more computation, more memory, and longer training time. Therefore, increasing the number of hidden neurons beyond an optimal point reduces the model's ability to generalize.

**Overfitting and Generalization**

- **Overfitting:** High training accuracy but low testing accuracy because the model learns noise in the training data.
- **Generalization:** The ability of the model to perform well on new, unseen data. Good generalization is achieved by selecting an appropriate number of hidden neurons and using techniques such as Dropout, L2 Regularization, Early Stopping, and Cross-Validation




**Conclusion**

Adding more hidden neurons increases the representational power of an MLP by enabling it to learn more complex patterns. However, too many neurons increase model complexity and may lead to overfitting, reducing generalization performance. Therefore, the number of hidden neurons should be carefully selected to balance learning capacity and generalization.

## 5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

**Question:**
Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

**Answer:**

A high-dimensional dataset contains a very large number of features. As the number of features increases, the model faces the curse of dimensionality, making it difficult to learn meaningful patterns. This increases computational complexity, slows training, and reduces the model's ability to generalize.

**Impact on Gradient Updates in Backpropagation**

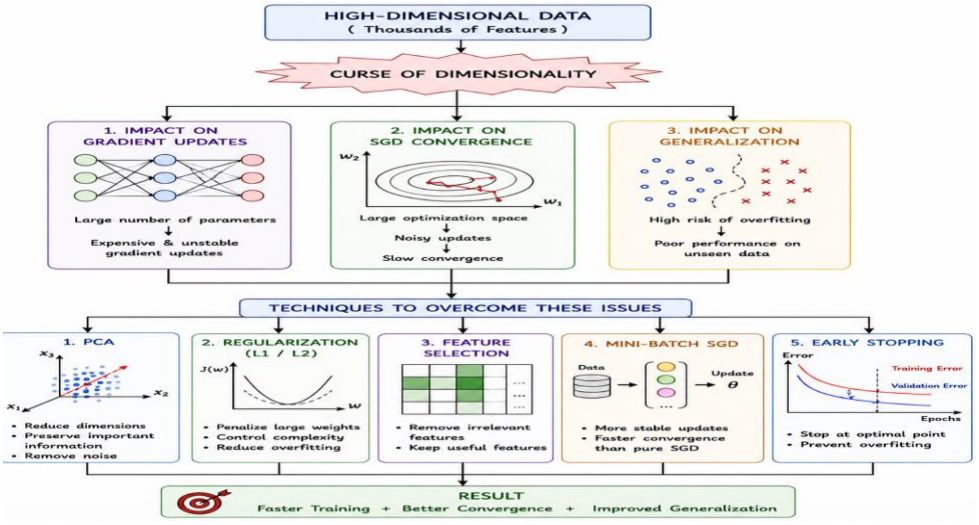
During backpropagation, gradients are computed for every parameter in the network. In high-dimensional data, the number of parameters becomes very large, making gradient computation more expensive. Many features may be irrelevant or noisy, causing unstable or inefficient gradient updates. This slows the optimization process and makes learning less effective.

**Impact on Convergence Speed of SGD**

Stochastic Gradient Descent (SGD) updates the model using one training sample at a time. With thousands of features, SGD requires more iterations to find the optimal solution because the optimization space becomes much larger. The gradients become noisy, leading to slower convergence and increased training time.

**Impact on Model Generalization**

High-dimensional data increases the risk of overfitting. The model may memorize the training data instead of learning general patterns, resulting in poor performance on unseen data. Consequently, the testing accuracy decreases even though the training accuracy remains high.




**Techniques to Overcome These Issues**

**1. Dimensionality Reduction (PCA)**

- Reduces the number of features while preserving the most important information.
- Removes noise and decreases computational cost.
- Improves convergence speed and model performance.

**2. Regularization (L1/L2)**

- Penalizes large weights and controls model complexity.
- Reduces overfitting and improves generalization.

**Other useful techniques:**

- **Feature Selection:** Removes irrelevant and redundant features.
- **Mini-Batch SGD:** Produces more stable gradient updates than pure SGD.
- **Early Stopping:** Prevents the model from overfitting by stopping training at the optimal point.

**Conclusion**

The curse of dimensionality negatively affects gradient computation, slows the convergence of SGD, and increases overfitting. Applying dimensionality reduction, regularization, feature selection, and Mini-Batch SGD improves training efficiency, accelerates convergence, and enhances the model's ability to generalize to unseen data.